

Reviewer Pack — Symmetric Power Multiplicity Gap in Permanent vs Determinant...

Entry 51256d50aa0d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 21:40:18 UTC

Symmetric Power Multiplicity Gap in Permanent vs Determinant Representations

Entry ID: 51256d50aa0d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 21:40:18 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Size for Permanent

Statement:

For a random $n \times n$ matrix A , the multiplicity of the trivial representation in the decomposition of $\text{Sym}^k(A)$ is exponentially larger for the permanent than for the determinant when $k = \Theta(n)$.

Rationale (proposer's reasoning):

The multiplicity of irreducible representations in symmetric powers relates to the structure of the underlying polynomial, which could encode computational complexity. The exponential gap in trivial representation multiplicities might reflect the inherent difficulty in computing the permanent compared to the determinant.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e7773b6e77626af5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: INCONCLUSIVE against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Schur-Weyl duality permanent circuit complexity multiplicity gap
- symmetric power decomposition trivial representation exponential gap - representation theory permanent lower bounds Schur-Weyl duality

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1701.06114v1] Geometric Schur Duality of two parameter quantum group of type A - [http://arxiv.org/abs/2006.13879v1] Two dualities: Markov and Schur-Weyl - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/1905.12360v2] The Monomial Lattice in Modular Symmetric Power Representations - [http://arxiv.org/abs/0711.0795v4] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields - [http://arxiv.org/abs/1603.02493v4] Clifford theory for glider representations - [http://arxiv.org/abs/math/0610591v3] Schur-Weyl duality in positive characteristic

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def binomial_coefficient(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))

def hook_length_formula(shape):
    rows, cols = len(shape), len(shape[0])
    total = 0
    for r in range(rows):
```

```

        for c in range(cols):
            total += shape[r][c] + 1 - r - c
    return binomial_coefficient(total, sum(sum(row) for row in shape))

def trivial_representation_multiplicity(n, k):
    # The multiplicity of the trivial representation in Sym^k(A) is given by the hook-length formula
    # applied to the partition (n-k, n-k, ..., n-k) repeated k times.
    shape = [[n - k] * k for _ in range(k)]
    return hook_length_formula(shape)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        permanent_multiplicity = trivial_representation_multiplicity(n, n)
        determinant_multiplicity = trivial_representation_multiplicity(n, n)

        if permanent_multiplicity < determinant_multiplicity:
            conjecture_holds = False
            counterexample = f"n={n}, perm_mult={permanent_multiplicity}, det_mult={determinant_multiplicity}"
            break

    total_metric_value += permanent_multiplicity / determinant_multiplicity
    instances_tested += 1

    return {
        "metric_name": "multiplicity_ratio",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}'\n first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture_
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture
TRIAL: {'metric_name': 'multiplicity_ratio', 'metric_value': 1.0, 'instances_tested': 6, 'conjecture

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_322a6872.py", line 88, in <module>
    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
                        ~~~~~^~~~~~

```

ZeroDivisionError: division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError due to empty results list, preventing evaluation of support fraction or counterexamples. | next: Fix the test code to handle empty results and re-run with sufficient sample size to validate conjecture

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	80453
2	preregistration	ollama_remote	qwen3:8b	0	0	24081
3	novelty	ollama_remote	qwen3:8b	0	0	16272
4	novelty	ollama_remote	qwen3:8b	0	0	24639
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15077

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9228
7	judge	ollama_remote	qwen3:8b	0	0	16050

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 185800 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/51256d50aa0d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/51256d50aa0d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/51256d50aa0d.tar.gz (if generated)